TermMap-DX: short write-up

HiLabs Hackathon 2026. The full documentation (architecture, protocol, error analysis, resources) is in `docs/WRITEUP.pdf`.

Approach and why

The labelled file has one row per (term, chunk, coding system), and **every term string is unique**, so the held-out terms are unseen strings. Most gold codes are long-tail: 40% of SNOMED rows have a gold concept that appears nowhere else in training. We therefore built a **retrieve-then-rank** engine that generalises from the supplied dictionaries and learns the annotators' convention from labelled neighbours, rather than memorising terms.

1. **Normalise:** markup, EHR artefacts, ICD codes written in the text, abbreviation variants (lexicon mined from the SNOMED file's own `ABBR - Expansion` descriptions), negation and history cues.
2. **Retrieve, per system:** exact key, char 3–4-gram TF-IDF (OCR noise), word TF-IDF, SapBERT, gte-base, Qwen3-Embedding, and **leave-one-out nearest labelled terms**. For ICD, SNOMED synonyms are added as query variants.
3. **Rank (stage 1):** LightGBM LambdaRank over ~85 features per candidate. Every signal scores every candidate, plus laterality, numbers/stage, specificity, semantic type × entity type, and code- and category-level neighbour votes.
4. **Re-rank (stage 2):** a fine-tuned cross-encoder (4 epochs), local **Qwen2.5-14B listwise log-probabilities** (one forward pass over a 10-option shortlist, with 5 retrieved coded examples in the prompt; deterministic) and ICD↔SNOMED coupling features.
5. **Output:** both codes for every row, isotonic-calibrated confidence and top-5 alternatives.

Results (5-fold CV on the 9,646 labelled rows; no leakage: LOO memory, shared folds)

	ICD-10-CM	SNOMED CT
exact match	**0.708**	**0.582** (diagnoses only: 0.669)
gold code unseen in training / seen 1–2×	0.457 / 0.704	0.443 / 0.614
50% most confident terms	0.881	0.762
right 3-character category	0.812	–
ceiling: label self-agreement on near-duplicate terms	*0.937*	*0.758*

The ladder of gains (min of the two systems) was: lexical 0.440, + SapBERT 0.500, + feature work 0.507, + two more encoders 0.515, + cross-encoder 0.535 (0.543 with 4 epochs), + LLM 7B 0.567, + LLM 14B **0.582**. Without an LLM the engine reaches 0.684 / 0.543 (0.78 s/term on CPU); the fully CPU set reaches 0.658 / 0.507 at 50 ms/term. If the LLM cannot run (no GPU, or a shared GPU without enough free memory), the engine falls back automatically (14B → 7B → no LLM) and says so. The same input gives byte-identical output on every run.

What we tried and abandoned

Hyper-parameter search, objective/seed ensembles and a diagnosis-only SNOMED ranker all moved CV by ±0.5 pt, so evidence, not capacity, is the bottleneck. A stage-2 ranker *without* new evidence slightly hurt, so stage selection is decided per system on out-of-fold accuracy. Using the client's own `gte_base` index as an oracle did not work (52% vs 61% for our ranker). BioLORD was dropped for licence reasons (requires UMLS/SNOMED licences). Context-only abbreviation disambiguation is correct on showcase cases but too rare in the training data (1.9% of rows) to move CV.

Where it still fails, and what next

- **The labels cap the score.** Near-identical terms share a gold code only 76% of the time in SNOMED (94% in ICD), so 95% exact match is not reachable on these labels. Adjudicating the ~80 conflicting duplicate pairs (and ~60 near-duplicate pairs) that `error_analysis.py` surfaces is the cheapest next step.
- **Retrieval misses** (8.7% ICD, 14.9% SNOMED) are paraphrases with convention targets ("hemodynamic instability" → R09.89). Next: a contrastively fine-tuned retriever on gold pairs.
- **Specificity** (8.4% ICD, same category, wrong leaf) and **semantic-type** confusions (13.3% SNOMED, procedure vs observable vs finding) are convention calls. Next: SNOMED IS-A hierarchy (RF2 release) for granularity features, and a written convention.
- **Cost.** Without the LLM: 50 ms–0.8 s/term on CPU. The LLM adds one forward pass per term per system and can be restricted to the low-confidence 30%.

External resources: the supplied dictionaries; SapBERT (Apache-2.0), gte-base (MIT), Qwen3-Embedding-0.6B (Apache-2.0), Qwen2.5-7B/14B-Instruct (Apache-2.0); LightGBM, scikit-learn, rapidfuzz, sparse_dot_topn. No UMLS, no official crosswalk, no hosted APIs.